

PROJECT DOCUMENTATION

Project Description

Program Overview

The Personal Finance Tracker is a Python application designed to help users manage their income, expenses, budgets, and savings goals. Through a menu-driven interface, users can record financial transactions, view summaries, set spending limits, and save their information for future use.

How the Program Works

The program is organized using multiple classes to separate different tasks. The Budget class stores monthly and weekly spending limits and checks whether expenses exceed the assigned budget.

The FinanceTracker class inherits from Budget, allowing it to use those budget features while also managing income, expenses, savings goals, and financial summaries. Users can add, modify, delete, save, load, and export transaction data through the main menu.

Algorithm Implementation

The program begins by creating a FinanceTracker object and checking for previously saved data. A menu loop continues running until the user chooses to exit, with each option performing a different financial task. When an expense is entered, the program compares it to the budget limit and displays a warning if the limit is exceeded. Financial summaries calculate total income, total expenses, net savings, and savings goal progress. Data is saved to a file, loaded when needed, and can be exported as a report. Basic error handling is also included when loading saved data.

Program Objectives

The program is designed to provide a simple way to organize personal finances. It allows users to track income, expenses, budgets, and savings goals while viewing summaries of their financial activity. Reports can also be exported for future reference.

User Interaction

Users interact with the program by selecting options from a numbered menu and entering transaction details such as amounts, categories, dates, and descriptions. Confirmation messages are displayed after completed actions, while invalid menu selections or file loading errors prompt users to try again.

Limitations

The current version runs in the command line, which may be less convenient than a graphical interface. Transactions must be entered manually, and the program cannot connect to bank accounts or automatically import financial information. It also does not include visual charts or graphs for analyzing spending.

Improvements

Future versions could include a graphical interface using tkinter to improve navigation and ease of use. Importing bank transactions would reduce manual data entry, while charts created with matplotlib could provide a clearer view of spending patterns. Additional features such as recurring bill reminders, password protection, support for multiple currencies, or database storage could further improve the program's functionality.

Pseudocode

PROGRAM PersonalFinanceTracker

```
// =====  
// CLASSES  
// =====
```

CLASS Budget:

This class stores spending limits for different categories

PROPERTIES: monthly_budget, weekly_budget

METHODS:

set_monthly_budget(category, amount)

set_weekly_budget(category, amount)

check_budget(category, amount)

CLASS FinanceTracker INHERITS FROM Budget:

This class inherits all the Budget properties and methods

PROPERTIES: income_list, expense_list, savings_goal

METHODS:

add_income(amount, category, date, description)

add_expense(amount, category, date, description)

set_savings_goal(amount)

get_summary()

```
// =====
```

```
// MAIN PROGRAM

// =====

START PROGRAM:

    PRINT "Welcome to Personal Finance Tracker"

    // Create the tracker object (uses inheritance from Budget)
    CREATE FinanceTracker object

    // Check if there is saved data
    IF file exists:
        ASK "Load saved data? (y/n)"
        IF yes:
            READ data from file
            PRINT "Data loaded"

    // Main menu loop
    REPEAT:
        PRINT "=====
        PRINT "1. Add Income"
        PRINT "2. Add Expense"
        PRINT "3. Set Savings Goal"
        PRINT "4. View Summary"
        PRINT "5. Modify Transaction"
        PRINT "6. Delete Transaction"
        PRINT "7. Save Data"
        PRINT "8. Load Data"
        PRINT "9. Export Report"
        PRINT "10. Set Budget"
```

```
PRINT "0. Exit"

PRINT "===== "

GET choice


IF choice == "1":
    // Add income
    PRINT "Enter amount"
    GET amount
    PRINT "Enter category (Salary, Business, Gift, etc.)"
    GET category
    PRINT "Enter date (YYYY-MM-DD)"
    GET date
    PRINT "Enter description"
    GET description
    ADD income to income_list
    PRINT "Income added"


ELSE IF choice == "2":
    // Add expense
    PRINT "Enter amount"
    GET amount
    PRINT "Enter category (Food, Housing, etc.)"
    GET category
    PRINT "Enter date (YYYY-MM-DD)"
    GET date
    PRINT "Enter description"
    GET description


// Using inherited Budget method to check spending limit
```

IF category has a budget limit:

IF current spending + new amount > budget limit:

PRINT "Warning: This will go over your budget!"

ADD expense to expense_list

PRINT "Expense added"

ELSE IF choice == "3":

// Set savings goal

PRINT "Enter savings goal amount"

GET savings_goal

SAVE savings_goal

PRINT "Savings goal set"

ELSE IF choice == "4":

// View summary

CALCULATE total_income = sum of all income

CALCULATE total_expenses = sum of all expenses

CALCULATE savings_balance = total_income - total_expenses

PRINT "===== FINANCIAL SUMMARY ====="

PRINT "Total Income: \$" + total_income

PRINT "Total Expenses: \$" + total_expenses

PRINT "Net Savings: \$" + savings_balance

IF savings_goal > 0:

CALCULATE progress = (savings_balance / savings_goal) * 100

PRINT "Savings Goal: \$" + savings_goal

PRINT "Progress: " + progress + "%"

```
PRINT "Income by Category:"  
FOR each income transaction:  
    PRINT category + ": $" + amount  
  
PRINT "Expenses by Category:"  
FOR each expense transaction:  
    PRINT category + ": $" + amount  
  
ELSE IF choice == "5":  
    // Modify transaction  
    PRINT "All transactions:"  
    FOR each transaction:  
        PRINT number + ". " + transaction details  
  
    PRINT "Enter number of transaction to modify"  
    GET transaction_number  
  
    PRINT "Enter new amount (or press Enter to skip)"  
    GET new_amount  
  
    PRINT "Enter new category (or press Enter to skip)"  
    GET new_category  
  
    PRINT "Enter new description (or press Enter to skip)"  
    GET new_description  
  
    UPDATE transaction with new information  
    PRINT "Transaction modified"
```

```
ELSE IF choice == "6":  
    // Delete transaction  
    PRINT "All transactions:"  
    FOR each transaction:  
        PRINT number + ". " + transaction details  
  
    PRINT "Enter number of transaction to delete"  
    GET transaction_number  
  
    PRINT "Are you sure? (y/n)"  
    GET confirm  
  
    IF confirm == "y":  
        DELETE transaction  
        PRINT "Transaction deleted"  
    ELSE:  
        PRINT "Deletion cancelled"  
  
ELSE IF choice == "7":  
    // Save data  
    OPEN file for writing  
    WRITE all income transactions to file  
    WRITE all expense transactions to file  
    WRITE savings_goal to file  
    WRITE budget limits to file  
    CLOSE file  
    PRINT "Data saved"
```



```
ELSE IF choice == "8":
```

```
    // Load data
```

```
    TRY:
```

```
        OPEN file for reading
```

```
        READ all income transactions from file
```

```
        READ all expense transactions from file
```

```
        READ savings_goal from file
```

```
        READ budget limits from file
```

```
        CLOSE file
```

```
        PRINT "Data loaded"
```

```
    CATCH error:
```

```
        PRINT "Error loading file"
```

```
ELSE IF choice == "9":
```

```
    // Export report
```

```
    OPEN file for writing
```

```
    WRITE "===== FINANCE REPORT ====="
```

```
    WRITE "Date: " + current_date
```

```
    WRITE "Total Income: $" + total_income
```

```
    WRITE "Total Expenses: $" + total_expenses
```

```
    WRITE "Net Savings: $" + savings_balance
```

```
    WRITE ""
```

```
    WRITE "All Transactions:"
```

```
    FOR each transaction:
```

```
        WRITE transaction details
```

```
    CLOSE file
```

```
    PRINT "Report exported"
```

```
ELSE IF choice == "10":
```

```
// Set budget (uses inheritance from Budget class)
PRINT "Enter period (monthly or weekly)"
GET period

PRINT "Enter category"
GET category

PRINT "Enter budget limit"
GET limit

IF period == "monthly":
    CALL set_monthly_budget(category, limit) // inherited method
ELSE:
    CALL set_weekly_budget(category, limit) // inherited method

PRINT "Budget set for " + category

ELSE IF choice == "0":
    // Exit
    PRINT "Save before exiting? (y/n)"
    GET save_choice

    IF save_choice == "y":
        SAVE data to file
        PRINT "Data saved"

PRINT "Goodbye!"
EXIT loop
```

ELSE:

PRINT "Invalid choice, try again"

UNTIL user chooses to exit

END PROGRAM